AWS VPC Peering Project

Connecting two Amazon VPCs over private IP addresses

Region	Asia Pacific (Sydney) — ap-southeast-2
Services	Amazon VPC, Amazon EC2
Date	29 October 2026
Peering connection	ewb-a-b-peer (pcx-03178b55064bce331)

1. Introduction

A Virtual Private Cloud (VPC) is an isolated network inside AWS. By default, two VPCs cannot exchange traffic privately. **VPC peering** is a networking connection between two VPCs that lets resources in either one communicate using private IPv4 addresses, as if they were on the same network. It needs no internet gateway, VPN or extra hardware.

2. Objectives

• Build two VPCs with non-overlapping CIDR blocks.

• Provide a public subnet with internet access in VPC A and a private subnet in VPC B.

• Deploy a bastion host in VPC A as the secure entry point.

• Request a VPC peering connection between VPC A and VPC B.

• Control access with route tables and security groups.

3. Architecture

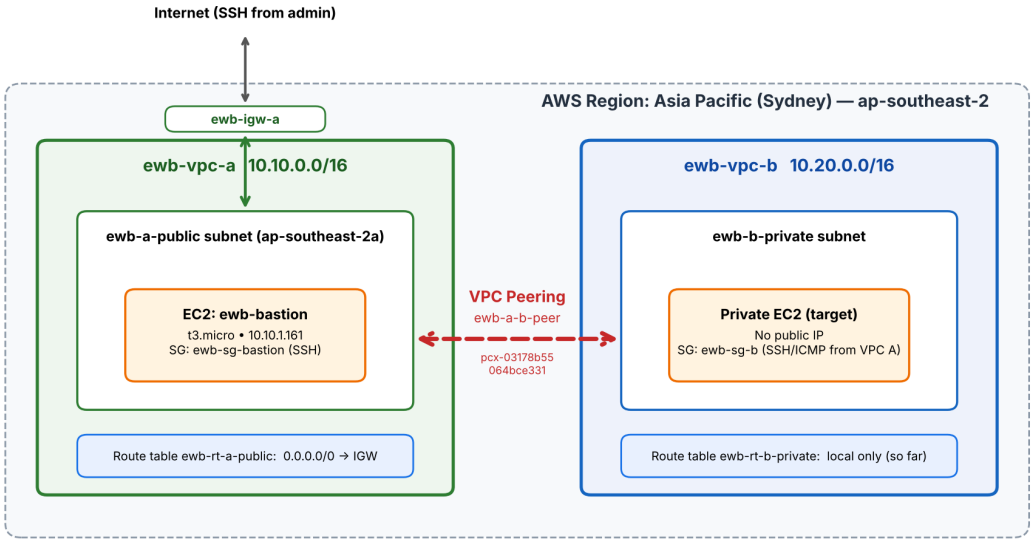


Figure 1 – Solution architecture

Component	VPC A	VPC B
VPC	ewb-vpc-a vpc-014c70892d7a0e4f8	ewb-vpc-b vpc-080c5958b2352dac7
CIDR	10.10.0.0/16	10.20.0.0/16
Subnet	ewb-a-public (ap-southeast-2a)	ewb-b-private
Route table	ewb-rt-a-public (rtb-03ecc3ca49b3892e8)	ewb-rt-b-private (rtb-00bf9d7a6174d98bc)
Internet gateway	ewb-igw-a (igw-056cb8e2394cefccb)	None
Security group	ewb-sg-bastion	ewb-sg-b
Compute	EC2 ewb-bastion, t3.micro, 10.10.1.161	Private EC2 (target)

4. Implementation

All steps were performed in the AWS Management Console in the Sydney region.

4.1 Open the VPC service

The VPC dashboard is reached by searching for VPC in the AWS console search bar.

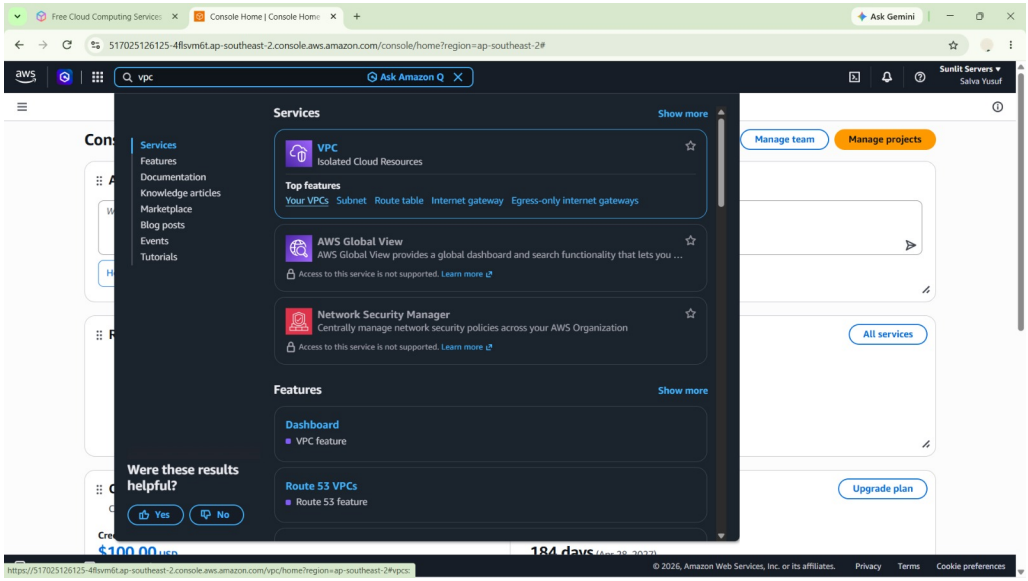


Figure – Searching for “VPC” in the console.

4.2 Create VPC A

Chose “VPC only”, named it **ewb-vpc-a**, set the IPv4 CIDR to **10.10.0.0/16**, no IPv6, default tenancy.

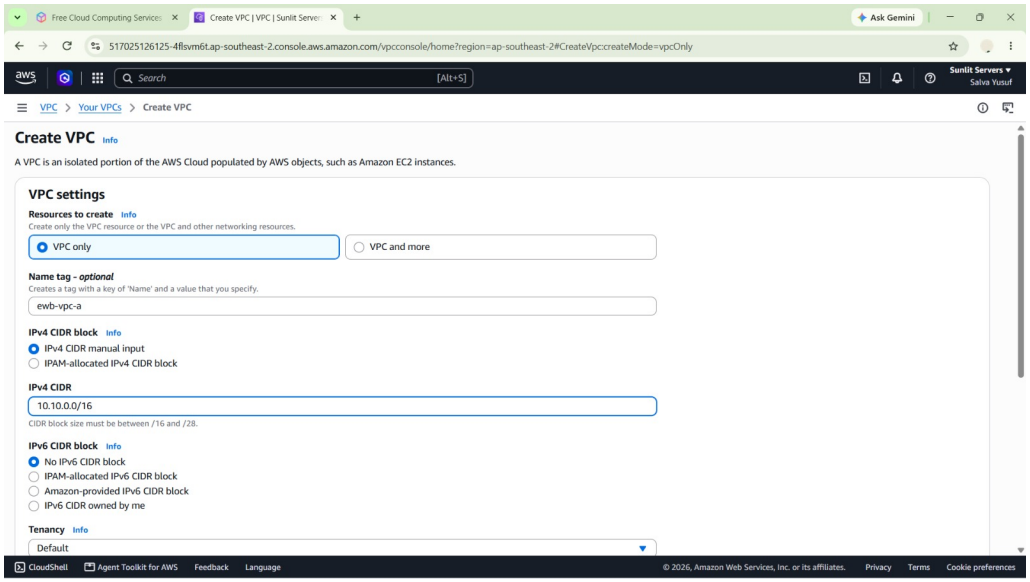


Figure – Create VPC form: name ewb-vpc-a, IPv4 CIDR 10.10.0.0/16.

AWS confirmed creation. DNS resolution is enabled, and the VPC is not the default VPC.

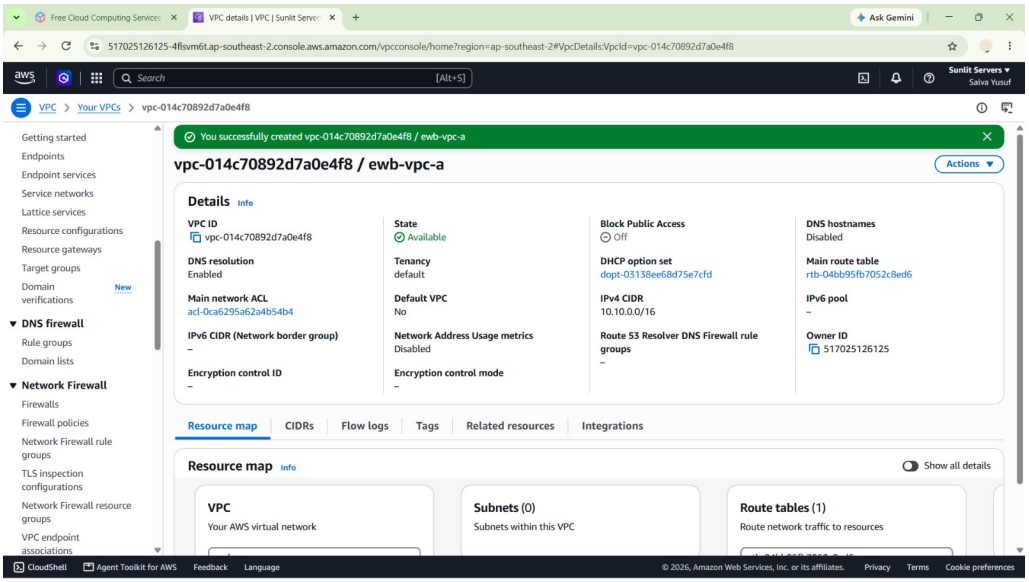


Figure – VPC A created and in the Available state.

4.3 Create the public subnet

A subnet named **ewb-a-public** was created inside VPC A in the ap-southeast-2a availability zone.

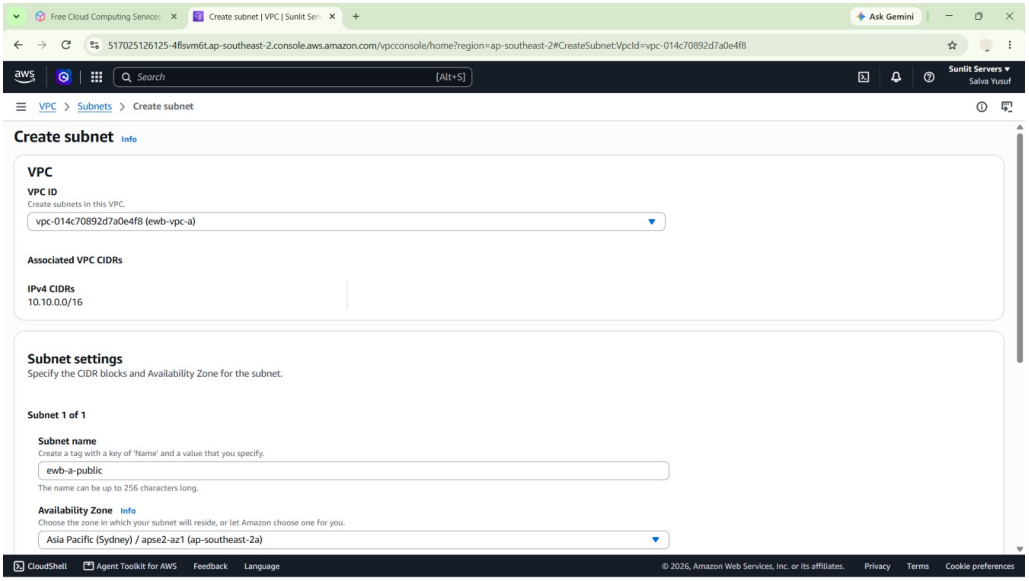


Figure – Subnet ewb-a-public in ewb-vpc-a, Availability Zone ap-southeast-2a.

4.4 Internet gateway

The internet gateway **ewb-igw-a** was created, then attached to VPC A so the public subnet can reach the internet.

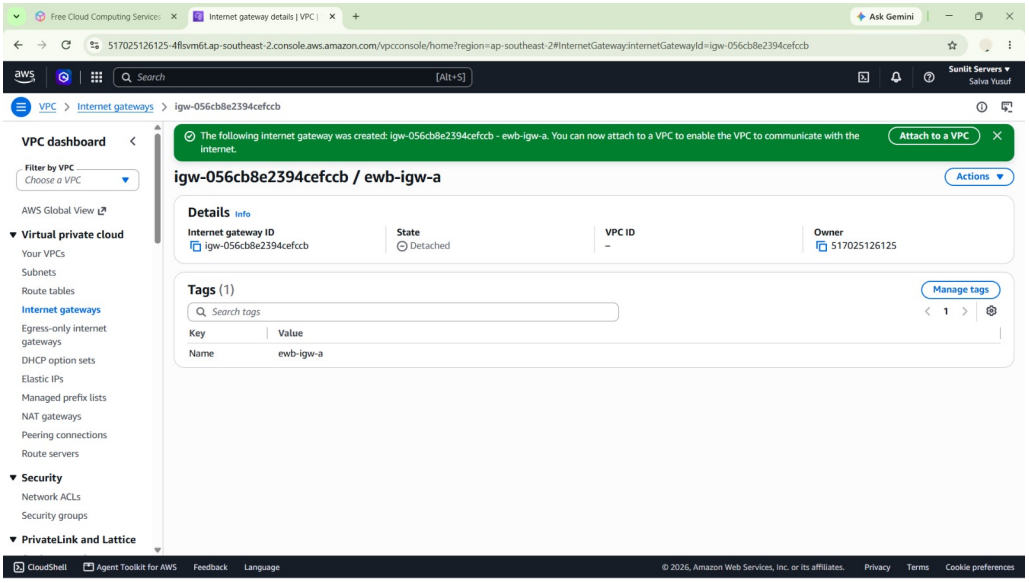


Figure – Internet gateway ewb-igw-a created (initially Detached).

4.5 Public route table

A custom route table was created for VPC A. It initially contains only the local route for 10.10.0.0/16.

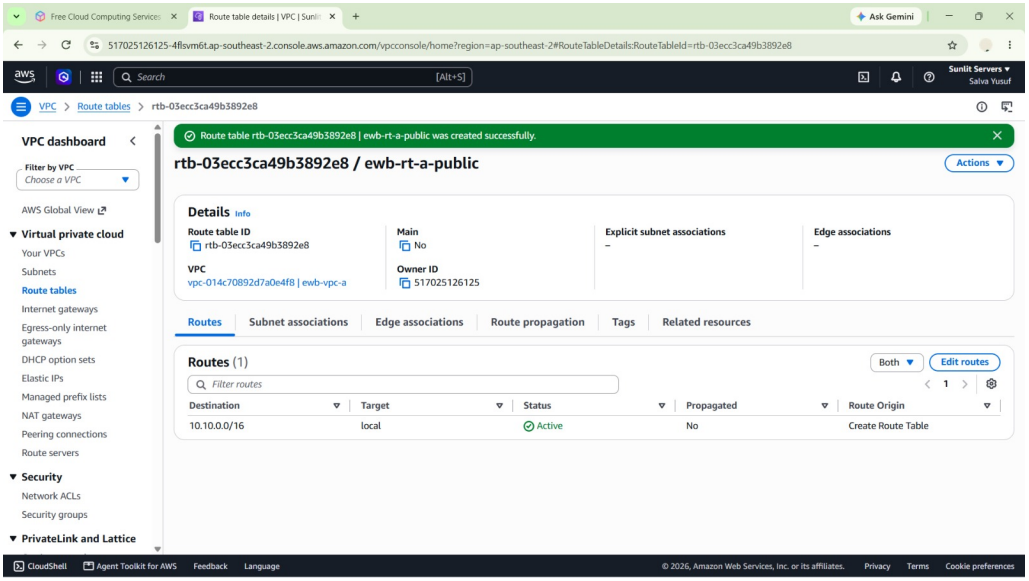


Figure – Route table ewb-rt-a-public with only the local route.

A default route to the internet gateway was added, and the route table was associated with ewb-a-public. This is what makes the subnet “public”.

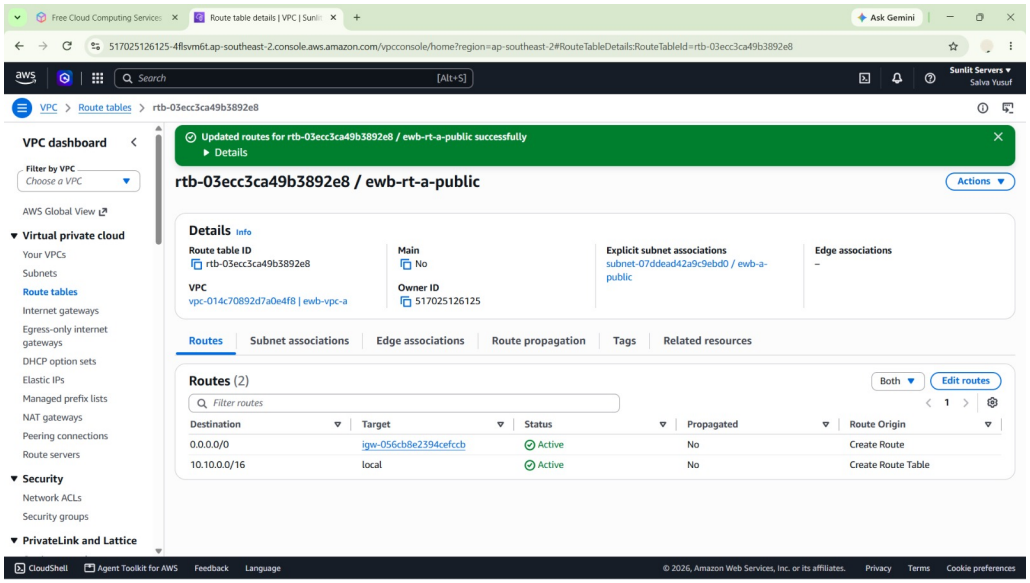


Figure – Route 0.0.0.0/0 → igw-056cb8e2394cefccb added and subnet associated.

4.6 Private route table (VPC B)

VPC B uses 10.20.0.0/16. Its route table has no internet route, so the subnet is private.

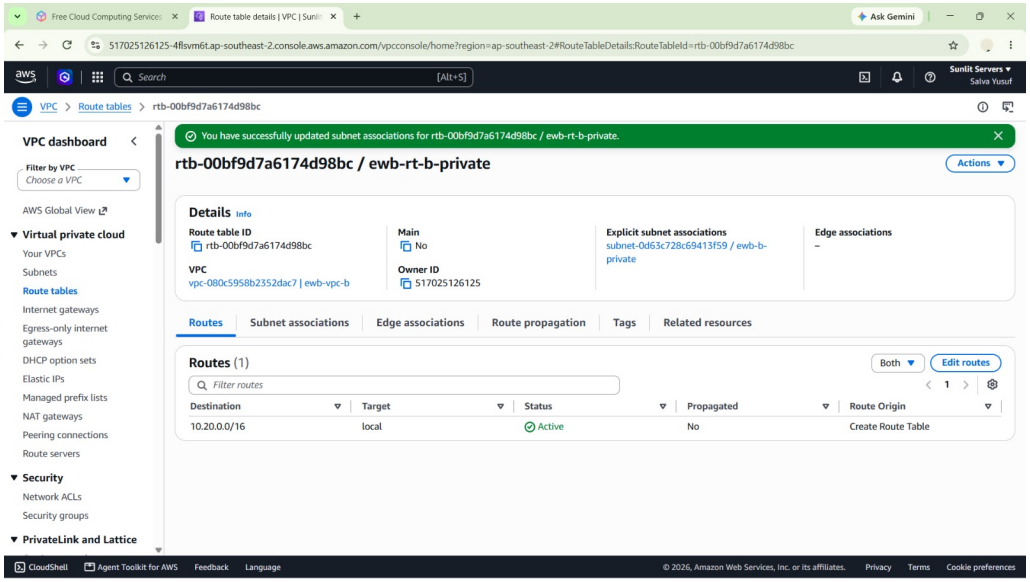


Figure – Route table ewb-rt-b-private associated with ewb-b-private; local route 10.20.0.0/16 only.

4.7 Security groups

A security group for VPC B was created. By default it has no inbound rules and one outbound rule.

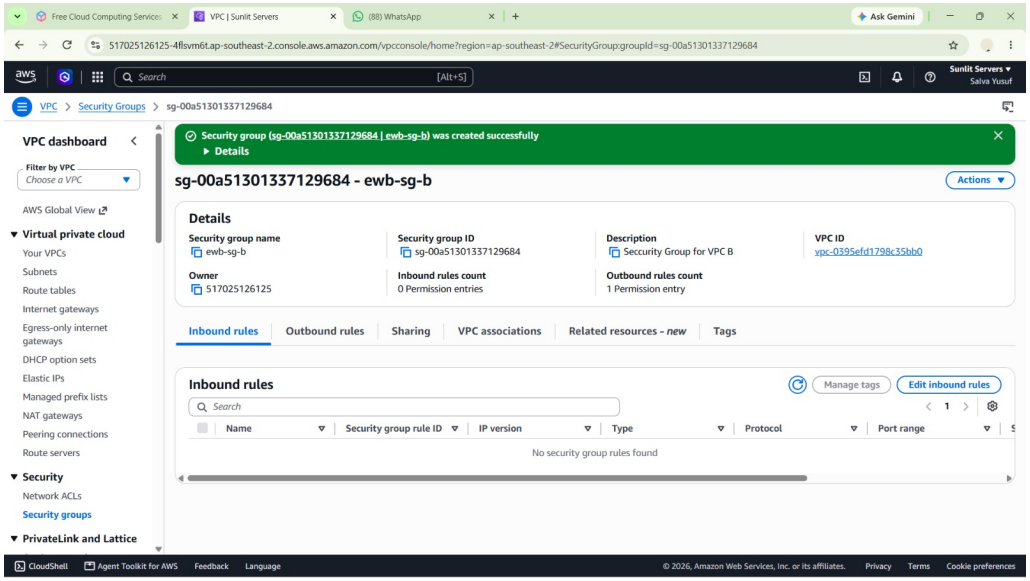


Figure – Security group ewb-sg-b created (no inbound rules yet).

Inbound rules were edited on ewb-sg-b. The list also shows **ewb-sg-bastion** in VPC A for SSH access to the bastion.

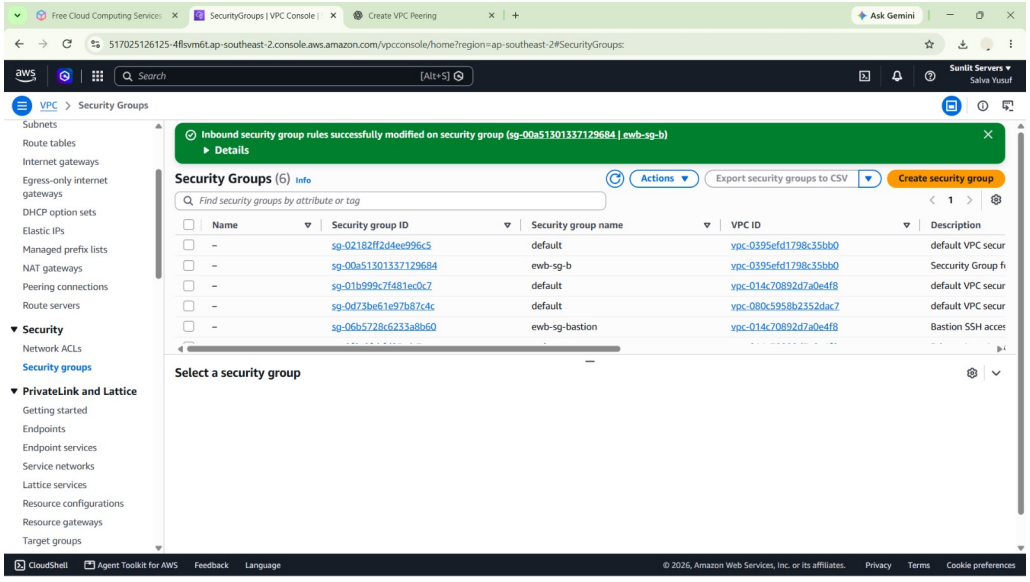


Figure – Inbound rules modified; security group list including ewb-sg-bastion.

4.8 Bastion host

The bastion host was launched in the public subnet of VPC A.

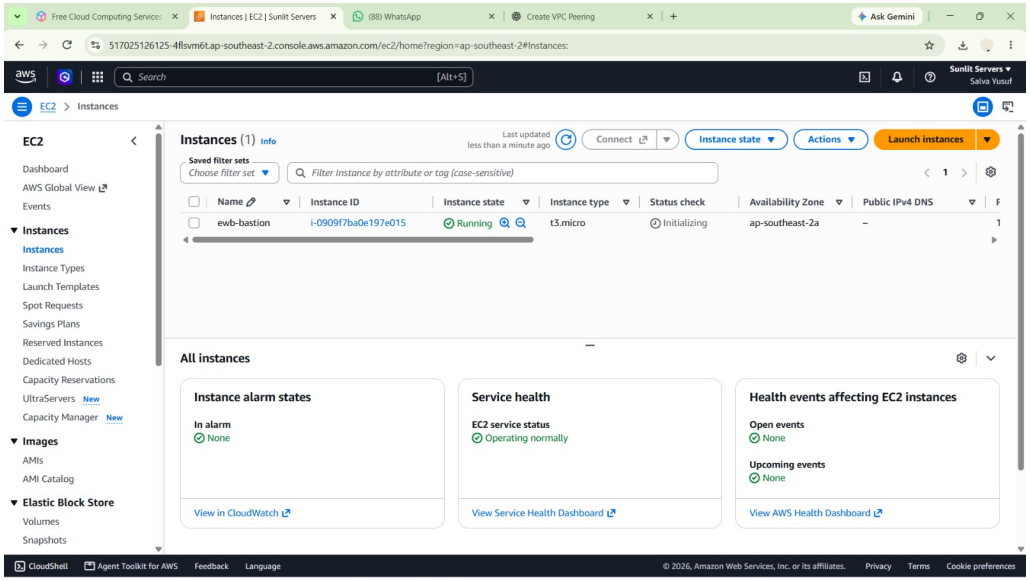


Figure – EC2 instance ewb-bastion (t3.micro) running in ap-southeast-2a.

SSH with a key pair authenticated successfully. The prompt shows the private IP 10.10.1.161, confirming the instance sits in VPC A's address range.

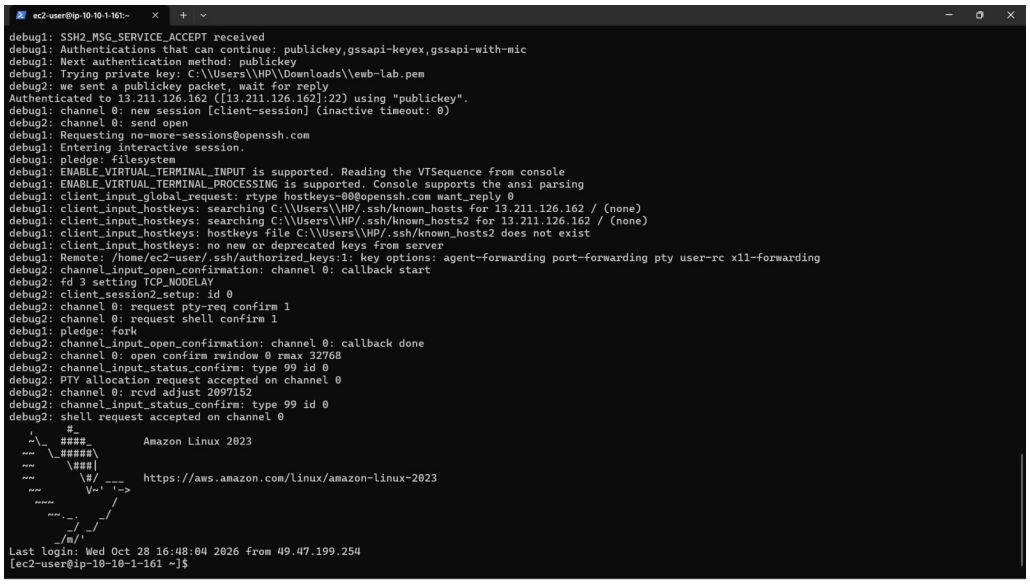


Figure – Successful SSH login as ec2-user (host ip-10-10-1-161).

4.9 VPC peering connection

A peering request was created from **ewb-vpc-a** (requester, 10.10.0.0/16) to **ewb-vpc-b** (accepter). Both VPCs belong to the same account and region, so the request can be accepted from the same console.

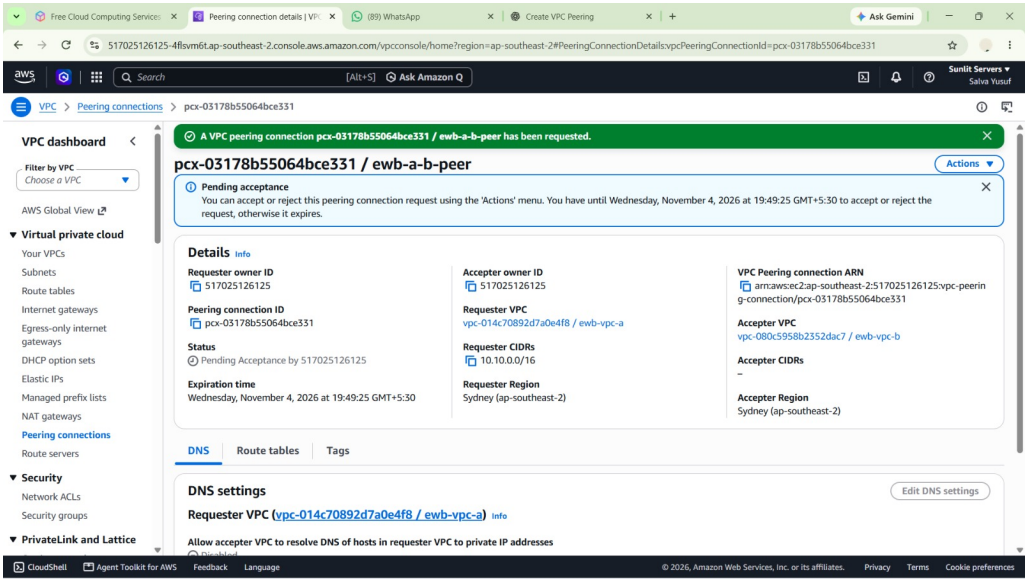


Figure – Peering connection ewb-a-b-peer requested; status Pending acceptance.

5. Status and remaining steps

The captured work covers VPC, subnet, gateway, routing, security group and bastion setup, plus the peering *request*. The peering connection was still **Pending acceptance** in the last screenshot, so end-to-end connectivity has not yet been demonstrated. To complete it:

• Accept the request: Peering connections → Actions → Accept request.

• Add **10.20.0.0/16** → **pcx-03178b55064bce331** to ewb-rt-a-public.

• Add **10.10.0.0/16** → **pcx-03178b55064bce331** to ewb-rt-b-private.

• Allow SSH (22) and ICMP from 10.10.0.0/16 in the security group of the private instance.

• From the bastion, run `ping <private IP>` and `ssh ec2-user@<private IP>` to verify.

6. Observation: security group placement

The security group list shows **ewb-sg-b** attached to `vpc-0395efd1798c35bb0`, not to ewb-vpc-b (`vpc-080c5958b2352dac7`). A security group can only be used by instances in its own VPC, so ewb-sg-b should be recreated inside ewb-vpc-b before launching the private instance.

7. Key learnings

• Peered VPCs must have non-overlapping CIDR blocks.

• Peering is not transitive and is configured per VPC pair.

• A peering connection alone is not enough: routes on both sides and permissive security groups are required.

• A subnet is “public” only because its route table points to an internet gateway.

• A bastion host limits the public attack surface to a single hardened instance.

8. Conclusion

Two VPCs with distinct address ranges were created in the Sydney region, with a public subnet and internet gateway in VPC A, a private subnet in VPC B, a working bastion host and a requested peering connection. Accepting the request and adding the cross-VPC routes and security group rules will complete the private connectivity between the two networks.